

A Report

on

Personal Finance Tracker (FinTracker)

carried out as part of the course Project Based Learning- 2/4

Submitted by

Piyush Prakash Sharma

2427030222

IVSemester

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

In

Computer Science & Engineering



**MANIPAL UNIVERSITY
JAIPUR**

Department of Computer Science & Engineering,

School of Computer Science and Engineering,

Manipal University Jaipur,

Jan-May 2026

A Report

on

Personal Finance Tracker (FinTracker)

carried out as part of the course Project Based Learning-2/4

Submitted by

Piyush Prakash Sharma

2427030222

IV Semester

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

In

Computer Science & Engineering

Under the Guidance of:

Guide Name : Ms. Harshika Mathur

Guide Signature (with date) : _____

Acknowledgement

This project would not have been completed without the help, support, comments, advice, cooperation and coordination of various people. However, it is impossible to thank everyone individually; I am hereby making a humble effort to thank some of them.

I acknowledge and express my deepest sense of gratitude to my internal supervisor Ms. Harshika Mathur for her constant support, guidance, and continuous engagement. I highly appreciate her technical comments, suggestions, and criticism during the progress of this project titled "Personal Finance Tracker (FinTracker)".

I owe my profound gratitude to Dr. Neha Chaudhary, Head, Department of CSE, for her valuable guidance and for facilitating me during my work. I am also very grateful to all the faculty members and staff for their precious support and cooperation during the development of this project.

Finally, I extend my heartfelt appreciation to my classmates for their help and encouragement.

Piyush Prakash Sharma

2427030222

Department of Computer Science and Engineering

School of Computer Science and Engineering

Date: _____

CERTIFICATE

This is to certify that the project entitled "Personal Finance Tracker (FinTracker)" is a bonafide work carried out as PBL-2 End Term Assessment in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering, by Piyush Prakash Sharma bearing registration number 2427030222, during the academic semester IV of year 2025-2026.

Place: Manipal University Jaipur, Jaipur

Name of the project guide: _____

Signature of the project guide: _____

Abstract

FinTracker is a personal finance and budgeting application designed to actively disincentivize overspending. It functions as a combination of a finance management app and a behavioral habit application — instead of rewarding users with cashbacks, it rewards them with psychological momentum through savings streaks. The system uses feedback loops and the psychological fear of losing that momentum to enforce accountability, helping users set financial goals and build genuine financial responsibility.

The application addresses a significant gap in the personal finance market: commercial payment apps (e.g., GPay, PhonePe) encourage spending through toxic positive reinforcement, while generic budgeting tools rely on passive visualization without creating emotional accountability. FinTracker bridges this gap by combining local, privacy-preserving data ingestion via CSV upload with a Machine Learning pipeline using a Naive Bayes NLP classifier to auto-categorize transactions, and a real-time React dashboard with behavioral feedback mechanisms.

PBL-1 delivered a functional Python script that parsed CSV bank statements, applied keyword-based categorization, and generated an Excel report with a pie chart. PBL-2 expanded this into a full-stack web application built with Flask and React, incorporating ML-powered categorization (TF-IDF + Naive Bayes), a real-time isometric dashboard with animated budget rings, and downloadable Excel reports with color-coded summaries.

Table of Contents

- 1 Introduction
 - 1.1 Objective of the Project
 - 1.2 Brief Description of the Project
 - 1.3 Technology Used
 - 1.3.1 Hardware Requirement
 - 1.3.2 Software Requirement
- 2 Design Description
 - 2.1 System Architecture / Flow Chart
 - 2.2 Data Flow Diagrams (DFDs)
 - 2.3 Comparative Study
- 3 Project Description
 - 3.1 Literature Review
 - 3.2 Libraries and Modules Used
 - 3.3 ML Pipeline Design
- 4 Input / Output Form Design
- 5 Testing & Tools Used
- 6 Implementation & Maintenance
- 7 Conclusion and Future Work
- 8 Outcome
- 9 Bibliography

1. Introduction

1.1 Objective of the Project

The objective of this project is to design and develop FinTracker — a personal finance management application that actively disincentivizes overspending. Unlike commercial payment applications that use toxic positive reinforcement to encourage spending, FinTracker employs behavioral psychology principles — specifically, savings streaks and passive-aggressive accountability notifications — to help users build genuine financial discipline.

The core goals are:

- To provide privacy-preserving, offline-first transaction tracking using local CSV uploads.
- To implement an ML-powered automatic expense categorization pipeline using TF-IDF vectorization and Naive Bayes classification.
- To create a real-time visual dashboard that dynamically reflects the user's budget status.
- To generate downloadable, formatted Excel reports with embedded charts for financial analysis.
- To implement a behavioral streak system that creates psychological accountability and encourages saving.

1.2 Brief Description of the Project

FinTracker is a full-stack web application that combines a Flask (Python) backend with a React.js frontend. The application accepts raw bank transaction data in CSV format, processes it through a machine learning pipeline, and presents the results on an isometric dashboard with animated budget rings and behavioral feedback.

The core concept is inspired by Duolingo's streak mechanism: instead of rewarding users financially (e.g., cashbacks), FinTracker rewards users with psychological momentum through savings streaks, and uses the fear of losing that streak to passively-aggressively notify users when they approach or exceed their budget.

The system was built across two phases. PBL-1 delivered a command-line Python script that performed keyword-based categorization and generated Excel reports. PBL-2 expanded this

into a complete web application with an ML-powered NLP classifier, a real-time behavioral dashboard, and a multi-channel output system.

1.3 Technology Used

1.3.1 Hardware Requirement

- Processor: Intel Core i3 or equivalent (minimum), Intel Core i5 recommended
- RAM: 4 GB minimum; 8 GB recommended for ML training
- Storage: 500 MB minimum for application and dependencies
- Display: 1280 x 720 resolution minimum; 1920 x 1080 recommended
- Internet: Required only for initial setup and dependency installation; all data processing is fully offline

1.3.2 Software Requirement

- Operating System: Windows 10/11, macOS 10.15+, or Ubuntu 20.04+
- Python 3.9 or higher
- Node.js 18+ and npm (for React frontend build)
- Flask 2.x (Python web framework)
- scikit-learn (Machine Learning pipeline: TF-IDF + Naive Bayes)
- pandas (Data ingestion, cleaning, and transformation)
- openpyxl (Excel report generation with embedded charts)
- joblib (Model serialization — saving/loading .pkl model files)
- Faker (Synthetic data generation for testing)
- React.js 18+ with Vite (Frontend dashboard)
- Web Browser: Google Chrome 90+ or Firefox 88+

2. Design Description

2.1 System Architecture / Flow Chart

The system follows a four-stage linear data pipeline architecture:

1. **Input Stage:** The user sets a monthly budget limit via the UI and uploads a raw, unstructured bank transaction CSV file. A Flask API endpoint (`/api/upload`) handles the local, offline data transfer.
2. **Processing Stage (ML Pipeline):** The raw data is cleaned by stripping currency symbols (₹) and handling negative values. Transaction text descriptions are then vectorized using TF-IDF, and the trained Naive Bayes NLP classifier predicts the exact expense category (e.g., Food, Utilities, Travel, Shopping, Subscriptions) without requiring manual keyword rules.
3. **Behavioral Analysis Stage:** The system calculates total spend versus the user-defined budget threshold. The emotional feedback engine evaluates the state of the user's Savings Streak and generates the appropriate pushback notification (from cautionary alerts at 75% budget usage to critical warnings beyond 100%).
4. **Output Stage:** The React frontend receives real-time JSON data and updates the isometric dashboard with animated SVG budget rings and category panels. Simultaneously, the `openpyxl` backend module generates a formatted, color-coded downloadable Excel report with embedded charts.

The architecture ensures complete privacy: all data processing occurs locally on the user's machine, with no data transmitted to any external server or cloud API.

2.2 Data Flow Diagram (DFD)

Level 0 — Context Diagram

The primary external entity is the User. The user provides two inputs to the FinTracker System: (1) a monthly budget limit and (2) a raw CSV bank statement. The system produces two outputs: (1) a real-time dashboard visualization and (2) a downloadable formatted Excel report.

Level 1 — DFD

At Level 1, the FinTracker system decomposes into four major processes:

5. **Data Ingestion & Cleaning Process:** Accepts the raw CSV input, strips invalid characters and currency symbols, validates column structure, and produces a cleaned dataframe.
6. **ML Categorization Process:** Takes the cleaned transaction descriptions, applies TF-IDF vectorization using the pre-trained vectorizer model, and predicts expense categories using the Naive Bayes classifier. Category labels are appended to the dataframe.
7. **Behavioral Analysis Process:** Calculates budget utilization percentage from the categorized dataframe and the user's budget limit. Outputs streak status and notification severity level.
8. **Output Generation Process:** Delivers two parallel outputs — JSON data to the React dashboard for live visualization, and an openpyxl-generated Excel workbook for download.

2.3 Comparative Study

Table 1: Comparative Analysis of FinTracker vs. Competing Solutions

Application	Cost	Privacy / Data Security	Categorization	Behavioral Focus
Cloud SaaS (YNAB, Monarch)	High Subscription	Cloud (Bank Sync Required)	Rule-based / Manual	Passive Tracking Only
Copilot Money	High Subscription	Cloud (Bank Sync Required)	AI / Machine Learning	Passive Tracking Only
UPI Apps (GPay, PhonePe)	Free	Cloud (Monetizes Data)	Basic / Vendor Level	Toxic Positivity (Encourages Spending)
Open Source (GnuCash)	Free	100% Local / Offline	Strictly Manual Entry	No Behavioral Guidance
FinTracker (Proposed)	Free	100% Local (CSV Upload)	ML-Powered (Naive Bayes)	Aggressive Accountability — Savings Streaks

3. Project Description

3.1 Literature Review

A comprehensive review of existing personal finance management solutions was conducted to identify the gap that FinTracker addresses.

1. Premium Cloud SaaS Tools (e.g., YNAB, Copilot Money): These platforms provide excellent interfaces and leverage machine learning for automatic transaction categorization. However, they are gated behind high annual subscription costs (averaging \$100/year) and require users to provide live bank credentials to a cloud service, creating significant privacy and security risks.
2. Open-Source and Local Tools (e.g., GnuCash, Microsoft Excel): These tools are entirely free and offer complete offline privacy. However, they impose extremely high friction on users through tedious manual data entry and offer no automation, making them impractical for daily use.
3. Modern UPI Applications (e.g., GPay, PhonePe): These are frictionless and highly convenient but employ 'toxic positive reinforcement' mechanisms — scratch cards, cashbacks, and reward points — that are deliberately designed to encourage continuous spending rather than saving.

The outcome of the literature review confirms that no existing solution simultaneously provides: (a) zero cost, (b) complete data privacy through local processing, (c) intelligent automated categorization, and (d) proactive behavioral accountability. FinTracker is designed to fill this exact gap.

3.2 Libraries and Modules Used

Machine Learning and AI

- `scikit-learn`: Core ML library used to build the TF-IDF Vectorizer (transforms transaction text descriptions into numerical feature vectors) and the Multinomial Naive Bayes Classifier (predicts expense category from the feature vectors).
- `joblib`: Used for model serialization. After the training script (`train_model.py`) completes training, `joblib` saves the trained models as `.pkl` binary files (`expense_model.pkl` and `expense_vectorizer.pkl`), enabling instantaneous loading at runtime without retraining.

Data Processing and Output

- pandas: Industry-standard Python library for data manipulation. Handles reading raw, unstructured CSV uploads, cleaning data (removing currency symbols, normalizing column headers), and preparing dataframes for the ML pipeline.
- openpyxl: Excel generation engine. Takes the categorized transaction data and produces a fully formatted .xlsx workbook containing a raw transactions sheet, a category summary sheet with color-coded rows, and an embedded pie chart.

Backend Framework

- Flask: Lightweight Python web framework that serves as the bridge between the Python ML backend and the React frontend. Exposes API endpoints including /api/upload (handles CSV ingestion) and /api/data (returns categorized JSON to the frontend).

Synthetic Data Generation

- Faker: Used exclusively in generate_large_csv.py to create realistic, randomized synthetic transaction datasets (fake merchant names, randomized amounts, realistic descriptions) to test application performance with 500+ row datasets without exposing real bank data.

3.3 ML Pipeline Design

The machine learning pipeline is a two-stage NLP classification system:

9. Text Vectorization (TF-IDF): Raw transaction description strings (e.g., 'UPI/Swiggy/Order') are converted into numerical TF-IDF (Term Frequency-Inverse Document Frequency) feature vectors. This approach captures the relative importance of words within the transaction corpus, enabling the model to distinguish between category-defining terms.
10. Classification (Multinomial Naive Bayes): The Naive Bayes classifier applies Bayes' theorem with a strong (naive) conditional independence assumption between features. This model was selected for its proven effectiveness in text classification tasks, low computational overhead (suitable for local deployment), and high interpretability.

The trained models are serialized using joblib and stored as .pkl files. The Flask application loads these files at startup, enabling real-time inference with negligible latency on each uploaded CSV.

4. Input / Output Form Design

4.1 Input Design

The application accepts two primary inputs from the user:

11. Budget Configuration Input: A numeric input field on the dashboard initialization screen where the user defines their monthly budget limit (in Indian Rupees). This value is stored in the application state and used as the threshold for all behavioral feedback calculations.
12. CSV Bank Statement Upload: A file upload interface that accepts a raw CSV file exported from any bank's internet banking portal or UPI application. The CSV must contain at minimum two columns: Description (transaction narrative) and Amount (transaction value in INR, with negative values for debits). The system sanitizes and normalizes the data automatically.

4.2 Output Design

The system generates two distinct output types:

13. Real-Time Dashboard (React Frontend): An isometric web dashboard featuring: (a) an animated SVG ring indicator displaying the percentage of budget consumed, with color transitions from green (safe) through amber (caution) to red (over budget); (b) a category breakdown panel listing all expense categories with their individual totals; (c) a recent activity feed showing the latest transactions; and (d) a persistent behavioral notification banner displaying the current savings streak status and contextual pushback messages.
14. Downloadable Excel Report (.xlsx): A multi-sheet Excel workbook generated by openpyxl containing: (a) a Transactions sheet with all categorized transactions, color-coded by category; (b) a Summary sheet with per-category spending totals; and (c) an embedded pie chart titled 'Spending by Category' visualizing the expenditure distribution.

5. Testing & Tools Used

5.1 Testing Methodology

The application was tested using a combination of synthetic data generation and manual functional testing across multiple budget threshold scenarios.

5.2 Synthetic Data Testing

The Faker library was used to generate a large test dataset (test_transactions.csv) containing over 500 rows of realistic, randomized bank transactions. This dataset included entries for all supported merchant categories (Swiggy/Zomato for Food, Uber/Ola for Travel, Amazon/Flipkart for Shopping, etc.) as well as edge cases such as refunds (positive amounts), salary credits, and international merchant names without standard category keywords.

Testing confirmed that the Naive Bayes classifier correctly categorized over 90% of synthetic transactions based on description text alone, without requiring manual keyword rules.

5.3 Budget Threshold Testing

The behavioral feedback system was verified across three distinct budget states:

15. Safe State (0–74% of budget consumed): Dashboard ring displayed in green; no warning notifications generated.
16. Caution State (75–99% of budget): Ring transitioned to amber; a caution banner appeared warning the user of impending budget exhaustion.
17. Over Budget State (100%+ consumed): Ring transitioned to red with a pulsing animation; a critical notification was displayed with a specific overspend amount and a directive to reduce discretionary spending immediately.

5.4 Tools Used

- Python 3.11 with VS Code IDE (backend development and ML training)
- Node.js 18 with Vite (React frontend development and bundling)
- Postman (API endpoint testing for /api/upload and /api/data routes)
- Faker library (synthetic test data generation)
- Microsoft Excel / LibreOffice Calc (verifying .xlsx report correctness)
- Google Chrome DevTools (frontend performance and network testing)

6. Implementation & Maintenance

6.1 PBL-1 Implementation (Phase 1)

The Phase 1 implementation consisted of a standalone Python script executing a linear data pipeline:

18. Input: The script accepted a raw CSV bank statement via command-line argument.
19. Cleaning: The `clean_amount_column()` function sanitized currency strings, removed ₹ symbols and commas, and standardized numeric data types using pandas.
20. Categorization: A keyword-matching algorithm using a predefined Python dictionary mapped transaction descriptions to expense categories (e.g., 'Swiggy' mapped to 'Food', 'Uber' mapped to 'Travel').
21. Reporting: The openpyxl library generated a structured .xlsx file containing a Transactions log sheet, a Category Summary sheet, and an embedded pie chart.

Phase 1 established the core data processing logic and validated the feasibility of CSV-based transaction analysis.

6.2 PBL-2 Implementation (Phase 2)

Phase 2 expanded the Phase 1 script into a complete full-stack web application:

22. Local Data Ingestion: A Flask web server was implemented to handle CSV uploads via a `/api/upload` REST endpoint, replacing the command-line interface with a browser-based UI.
23. ML Pipeline Integration: The keyword-matching categorizer was replaced with a trained Naive Bayes NLP classifier. Transaction descriptions are vectorized using TF-IDF before classification, enabling the system to correctly categorize transactions it has never seen before.
24. Behavioral Analysis Engine: A budget utilization calculator was implemented that computes the percentage of the user-defined monthly budget consumed and triggers the appropriate streak/notification state.
25. React Dashboard: A React.js frontend with animated SVG components was built, receiving real-time categorized data from the Flask API via JSON responses and rendering the isometric budget ring, category panels, and activity feed.

26. Excel Report Generation: The openpyxl report generator from Phase 1 was integrated as a backend module, producing color-coded, chart-embedded downloadable reports.

6.3 Maintenance Considerations

- **Model Retraining:** The Naive Bayes classifier can be retrained by running `train_model.py` with an updated labeled dataset. New merchant categories can be added without modifying application logic.
- **CSV Format Compatibility:** The cleaning pipeline is designed to handle multiple CSV formats from different Indian banks. Edge cases (negative amounts for debits vs. credits, different date formats) are handled by the `clean_amount_column()` normalization function.
- **Dependency Management:** All Python dependencies are defined in `requirements.txt`. Node.js dependencies are managed via `package.json`. The application is fully portable across operating systems.

7. Conclusion and Future Work

7.1 Conclusion

FinTracker successfully demonstrates a privacy-first, behavioral-psychology-driven approach to personal finance management. By replacing the passive pie charts of traditional budgeting tools with active streak-based accountability mechanisms, and by eliminating the toxic positive reinforcement patterns of commercial UPI applications, FinTracker provides a genuinely differentiated solution for the Indian personal finance market.

The transition from a simple keyword-based Python script in PBL-1 to a full-stack ML-powered web application in PBL-2 validates the technical feasibility of local, offline-first intelligent finance tracking. The Naive Bayes NLP classifier demonstrated high categorization accuracy on synthetic test data, and the Flask-React architecture provides a clean separation of concerns between the ML backend and the visualization frontend.

The project successfully achieves all stated objectives: zero-cost operation, complete data privacy through local processing, automated ML-powered categorization, real-time behavioral feedback, and downloadable formatted reports.

7.2 Future Work

The following enhancements are planned for future development phases:

27. WhatsApp Bot Integration for Frictionless Manual Entry: A conversational AI bot deployed via the WhatsApp Business API will allow users to log offline or cash transactions by simply texting a message such as 'Spent ₹300 on auto' to the bot. The message is parsed and passed to the Naive Bayes classifier for categorization. This feature addresses behavioral friction — users already spend hours daily on WhatsApp, so moving the input mechanism there eliminates the need to open a separate application.
28. Real-Time Device Notification and SMS Parsing: Transitioning from static CSV uploads to a local on-device listener that intercepts and parses bank transaction SMS messages and UPI push notifications in real time. Immediate feedback drives stronger behavioral change — by capturing transactions at the moment of occurrence, the app can instantly trigger savings streak alerts and pushback notifications, closing the psychological gap between the act of spending and the awareness of budget impact.

29. Deep Learning Classifier Upgrade: Replacing the Naive Bayes classifier with a fine-tuned BERT or DistilBERT model to improve categorization accuracy for ambiguous or multi-language transaction descriptions common in Indian UPI payments.
30. Multi-User and Family Budget Support: Extending the application to support multiple user profiles with shared budget pools for family or household expense tracking.

8. Outcome

Current Status: Functional full-stack web application prototype (PBL-2 complete). The application is deployable locally and has been tested with synthetic datasets of 500+ transactions.

Research Paper: A research paper documenting the behavioral finance approach, the ML categorization pipeline, and the comparative effectiveness of streak-based accountability vs. passive visualization is in preparation for submission to an undergraduate research journal.

Deployment: The application is designed for local deployment as a personal tool. Future work includes packaging the application as a standalone Electron desktop application for one-click installation, and submission to an app store upon integration of the SMS parsing and WhatsApp bot features.

9. Bibliography

- [1] Pandas Development Team. (2024). pandas: powerful Python data analysis toolkit. Available: <https://pandas.pydata.org>
- [2] Gazoni, E. & Clark, C. (2024). openpyxl: A Python library to read/write Excel 2010 xlsx/xlsm files. Available: <https://openpyxl.readthedocs.io>
- [3] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
- [4] Plaid Inc. (2024). Plaid API Documentation. Available: <https://plaid.com/docs/>
- [5] You Need A Budget (YNAB). (2024). The Four Rules of Budgeting. Available: <https://www.ynab.com/the-four-rules>
- [6] Flask Development Team. (2024). Flask: A lightweight WSGI web application framework. Available: <https://flask.palletsprojects.com>
- [7] React Team. (2024). React: A JavaScript library for building user interfaces. Available: <https://react.dev>